OAuth 2.0

limited, revocable access without your password

Watch the 4-min video

In one line: OAuth 2.0 is a **delegation** protocol: the user approves limited scopes at the authorization server, and the app receives an access token, never the user's password; use the authorization code flow with PKCE for users and client credentials for services.

1 Sharing passwords

A photo-printing app wants your Google Drive photos. The old way: give it your Google password. Now it can read your email and change your password, and you can't take access back without changing the password everywhere. OAuth 2.0 (RFC 6749) fixes this: the app gets a token that is limited in **scope** and **time** and can be **revoked**.

2 Four roles

The **resource owner** is you. The **client** is the print app. The **authorization server** (Google's login and consent service) authenticates you, shows consent and issues tokens. The **resource server** (the Drive API) accepts tokens and enforces their scopes. The key idea: the client gets an access token, never your credentials.

3 The authorization code flow

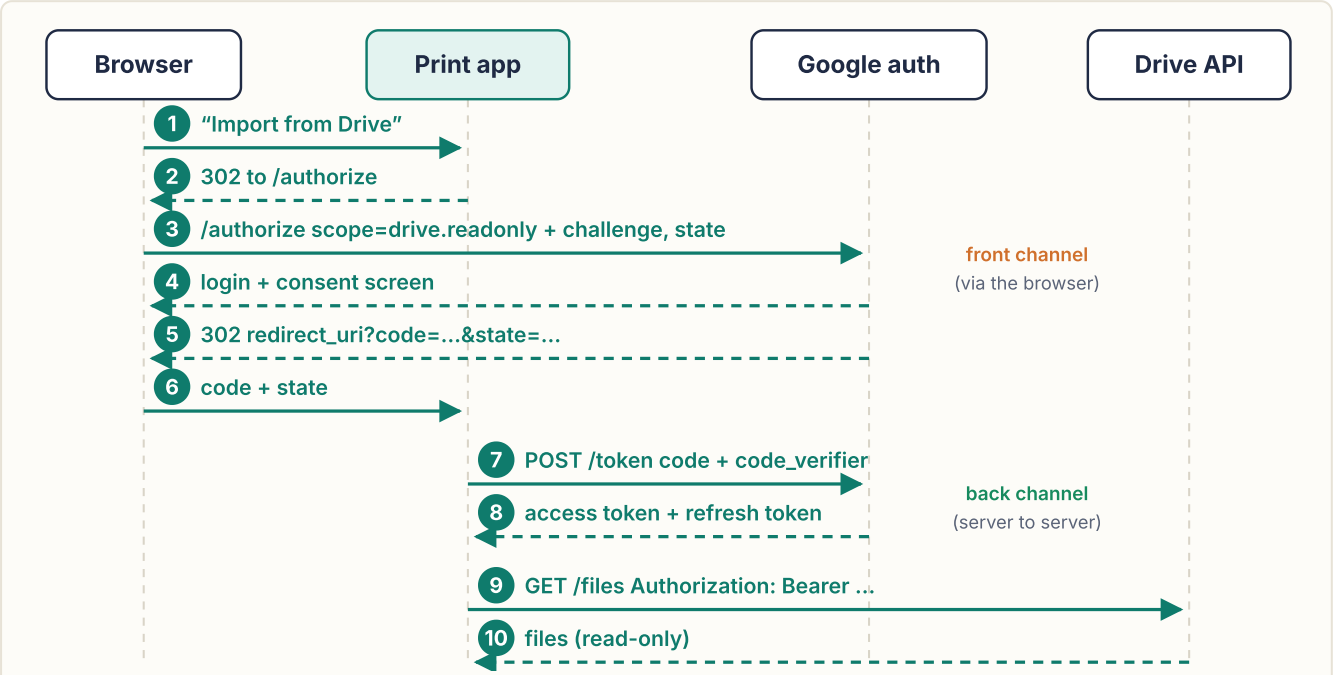


Figure 1. Authorization code flow with PKCE. Only a short-lived, single-use code passes through the browser; tokens travel on the back channel.

The app redirects the browser to the authorization endpoint with `response_type=code`, its `client_id`, a pre-registered `redirect_uri`, the `scope`, a random `state` (checked on return, against CSRF) and the PKCE challenge. After login and consent the server redirects back with a **code**: single-use, short-lived

(RFC 6749 recommends at most 10 minutes). The app's back end exchanges it at the token endpoint. The redirect URI must match exactly, or codes could be sent to an attacker's site.

4 PKCE

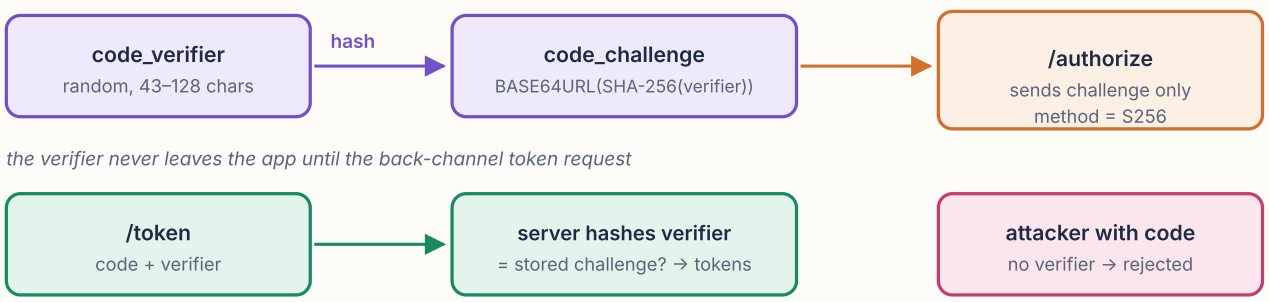


Figure 2. PKCE (RFC 7636, “pixy”): a stolen authorization code is useless without the verifier that only the real app holds.

On mobile, another app can register the same custom URL scheme and intercept the code; PKCE binds the code to the app that started the flow. Always use `S256`, not `plain`. The OAuth 2.0 Security Best Current Practice (RFC 9700, January 2025) recommends PKCE for **every** client, including confidential web apps with a client secret.

5 Scopes, refresh and revocation

Scopes limit what a token can do: `drive.readonly` instead of full Drive access, shown on the consent screen. Access tokens are short-lived (Google's last an hour); the app then sends its **refresh token** (`grant_type=refresh_token`) for a new one silently. The user can revoke the app at any time, which kills the refresh token.

6 Which flow?

Client	Flow	Notes
Web app with a back end	authorization code + PKCE	confidential client: also authenticates with a secret or certificate
SPA, mobile, desktop app	authorization code + PKCE	public client: can't keep a secret, so PKCE is essential
Service to service, cron job	client credentials	no user; the service gets a token for itself
TV, CLI, IoT (no browser)	device authorization (RFC 8628)	user approves on a phone with a short code
Implicit · password grant	don't use	implicit put tokens in the URL; password grant hands the app the password

WORKED EXAMPLE: IMPORTING FROM GOOGLE DRIVE

The print app stores a fresh verifier in the user's session and redirects to Google's authorization endpoint with `scope=…/auth/drive.readonly`, `code_challenge`, `code_challenge_method=S256` and `state`. The user approves; Google redirects back with a code. The app checks `state`, POSTs code + verifier to the token endpoint, and calls the Drive API with `Authorization: Bearer …`. The user can revoke it in their Google Account at any time.

COMMON MISTAKES

- Using an OAuth access token as proof of login: that's what OpenID Connect is for (lesson 43).
- Loose redirect URI matching (wildcards, prefixes), which lets attackers receive codes.
- Skipping `state` or PKCE; using the deprecated implicit flow in a new SPA.
- Requesting broad scopes “just in case”, or shipping a client secret inside a mobile app.

7 Key terms

Authorization code: single-use, short-lived code exchanged for tokens

Scope: named permission the token carries

Public vs confidential client: can't vs can keep a client secret

PKCE: verifier + hashed challenge that bind the code to the app

Refresh token: used at the token endpoint to get new access tokens

Client credentials: grant for a service acting as itself

8 Interview questions

Q1. What are the four OAuth 2.0 roles? BASIC

Resource owner (the user), client (the app), authorization server (issues tokens after login and consent) and resource server (the API that accepts tokens).

Q2. Which flow is used for service-to-service calls? BASIC

Client credentials: the service authenticates as itself with its client ID and a secret or certificate, and gets a token with no user involved.

Q3. Walk through the authorization code flow with PKCE. SCENARIO

The client creates a random verifier and sends its SHA-256 hash (the challenge) when redirecting the user to the authorization server. The user logs in and consents; the server redirects back with a short-lived code. The client's back end exchanges code + verifier at the token endpoint for tokens.

Q4. You're building a React SPA that calls your API. Which flow? SCENARIO

Authorization code with PKCE as a public client (no secret), or better a backend-for-frontend that does the flow server side and keeps tokens out of the browser.

Q5. Why was the implicit flow deprecated? TRICKY

It returned the access token in the URL fragment, where it leaks via history, referrers and logs and can't be bound to the client. Code + PKCE gives the same browser support without those leaks.

Q6. What does the state parameter protect against, and does PKCE replace it? TRICKY

state stops CSRF on the redirect (an attacker injecting their own code into your session). PKCE also prevents that when the server enforces it, but state is still used to carry and check request context.

Next: SD-M-005 · OpenID Connect and SSO backendroadmap.com